

Prezados Alunos (as), boa noite.

O roteiro desse módulo se refere à implementação dos conceitos:

- 1) Demonstração no EA sobre como criar diagrama de sequência;
- 2) Demonstração no EA sobre como criar diagrama de casaco de uso;
- 3) Interface em OO;

Siga as instruções abaixo para completar o roteiro desse módulo:

- 1) Assista ao seguinte vídeo de apoio ao conteúdo:
<http://>

- 2) Leia sobre o conceito de interface em OO e C# no link abaixo:
<https://learn.microsoft.com/pt-br/dotnet/csharp/programming-guide/interface-s/explicit-interface-implementation>

- 3) Crie um projeto com o nome ProjetoTesteInterface;

Inclua no seu projeto a interface IfcFigura com a seguinte estrutura:

```
public interface IfcFigura{
    public abstract static void calcularArea();        // Calcula a área
    de uma figura
    public abstract static void imprimirDados();      // Imprime a área
    de uma figura
}
```

- 4) Incorpore os fontes do diretório exemplos ao projeto
ProjetoTesteInterface

- 5) Mude a implementação da classe FiguraVO para implementar a interface
IfcFigura

- 6) Crie outra interface com o nome IfcContratoFigura com a seguinte
estrutura:

```
public interface IfcContratoFigura{
    public abstract static int retornarNumeroObjetosFigura();    //
    retorna o número de instâncias do tipo figura
}
```

Obs: crie um atributo estático para controlar a quantidade de objetos do
tipo Figura instanciados;
Qual o melhor local para incrementar o atributo, independentemente do tipo
de figura instanciado?

- 7) Edite a classe Figura para ter um atributo nomeado idFigura do tipo
inteiro e que deverá ser inicializado no construtor com o valor número 1,
se foi o primeiro objetos instanciado; 2 se foi o segundo e assim por
diante.

- 8) Edite o método imprimirDados das classes para retornar o idFigura dos
objetos;

- 9) No main do seu projeto faça testes com as novas implementações para
assegurar o entendimento dos conceitos estudados;

PARTE 2: Teste de polimorfismo;

- 10) Incorpore a classe Retangulo abaixo no seu projeto:

```
public class Retangulo:Figura
{
    private double comprimento;
    private double altura;
```

```
public Retangulo(int codigo, string descricao, double comprimento,
double altura) : base(codigo, descricao)
{
    this.codigo = codigo;
    this.descricao = descricao;
    this.comprimento = comprimento;
    this.altura = altura;
}

public override double calcularArea()
{
    return (comprimento*altura);
}
public override void imprimirDados()
{
    base.imprimirDados();
    Console.WriteLine("\nComprimento: " + comprimento+" \nAltura:
"+altura);
}
}
```

11) Refatore a classe Retangulo para RetanguloVO

12) Na última linha do Main, instancie o objeto nomeado listaDeObjetos do tipo List<FiguraVO> com 10 posições. Popule o listaDeObjetos de forma que as posições pares receberão objetos do tipo RetanguloVo com valores de atributos aleatórios e nas posições ímpares com objetos do tipo CirculoVO;

13) Inclua um código para iterar no objeto listaDeObjetos e chamar o método calcularArea e imprimirDados de cada posição;

14) O que o teste da etapa 13 tem a ver com polimorfismo?

Faça outros testes que julgar necessário e leve/esclareça as eventuais dúvidas com o Professor.

Bom trabalho,

Geraldo.